

NURobotics Firmware Course

Week 1: Intro to Firmware & C/C++ Programming

Who Am I?

- Casey Goyette
- PlusOne Master's student in Electrical & Computer Engineering w/ Concentration in Hardware & Software for Machine Intelligence
- VEX U software lead
- Co-ops/Internships
 - Electrical Engineering @ ColdSnap
 - Firmware Engineering @ Fleet Robotics
 - Embedded Software @ NK Labs
 - Embedded Software @ Bose



Course Logistics

- Head to `#course-roles` to assign yourself the firmware class role and get access to the `#firmware-course` channel
- This course will be held every Monday from 6:30PM – 8:00PM (except university holidays) in Richards 458
 - 9/21, 9/28, 10/5, 10/19, 10/26, 11/2, 11/9, 11/16
- Any room/time changes will be communicated in `#firmware-course`
- Course format:
 - Each class will start with a lecture and end with a lab

Course Overview

- **Week 1: Intro to Firmware & Programming**
- Week 2: Analog I/O & Interrupts
- Week 3: Communication Protocols
- Week 4: Driver Development
- Week 5: State Machines + RTOSes
- Week 6: STM32 Fundamentals
- Week 7: STM32 Peripherals
- Week 8: Communication and RTOS Capstone

A Note on AI Usage in Embedded Development

Yes, Claude could probably do most of these labs for you. If your goal is to have Claude write the code, you're probably in the wrong course. This is embedded development, not web dev.

In general, AI is not very good at embedded development.

Good AI Usage

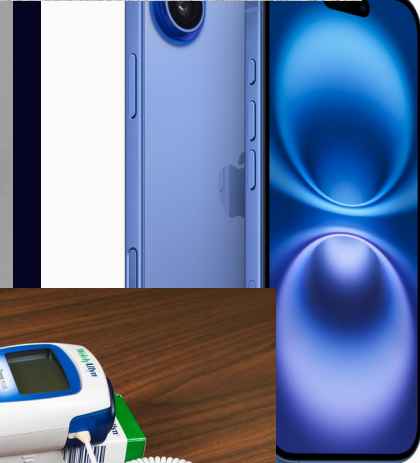
- Debugging
- Filtering through documents/datasheets
- Code snippet generation

Bad AI Usage

- Writing entire files
- Reformatting anything
- Most firmware things

What is Firmware?

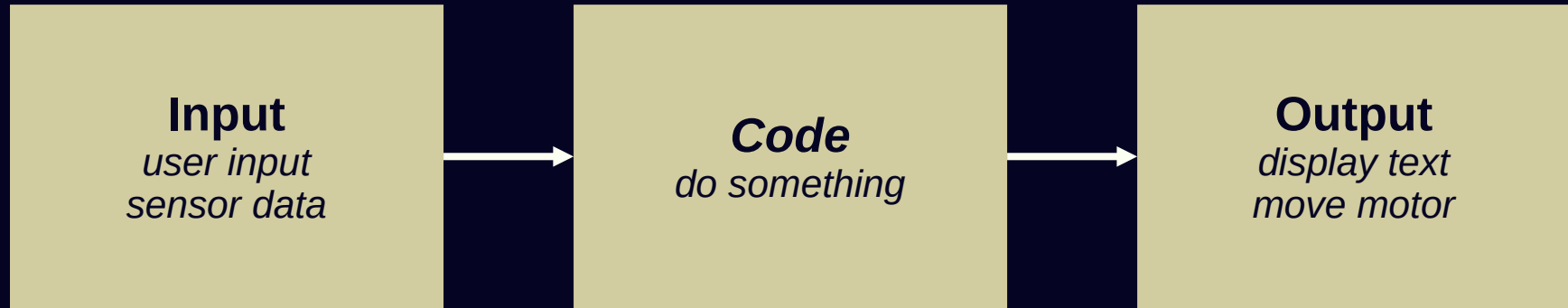
“Software For Hardware”



Firmware

- Software vs Firmware
 - Firmware directly controls the functionality of devices at the hardware level
 - Software improves user interaction with devices
- Firmware is the bare metal software embedded pretty much every modern electronic device
 - Ever had to install a driver on your computer (wifi, GPU, mouse, etc.)???
- Firmware is generally designed to run on a specific device to do a specific thing
 - Code needs to be compiled for specific targets
- Most common languages:
 - Assembly
 - C/C++ <-- mostly this
 - Rust
 - Python

What is Code?



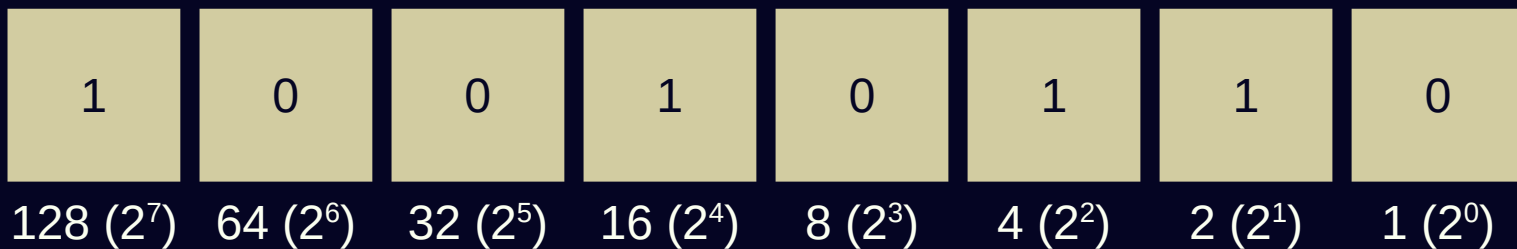
It's all binary!

Modern computers are digital devices, which means that they operate on 1s and 0s



The Byte

Smallest addressable unit of memory in many computer architectures

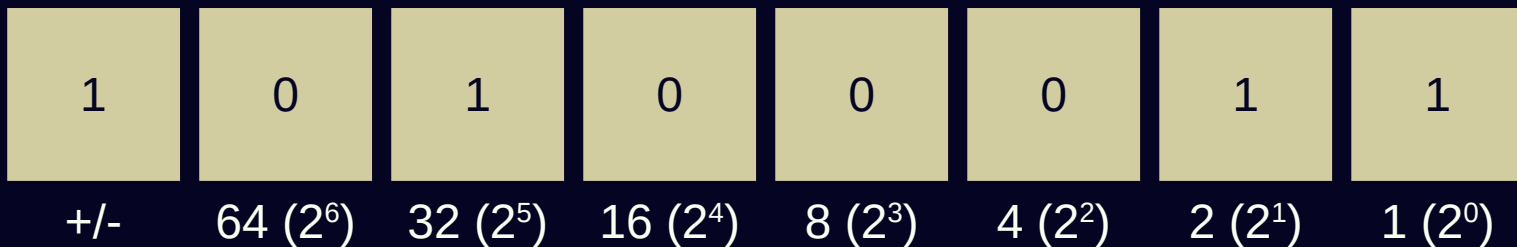


The Byte

1	0	0	1	0	1	1	0
128 (2^7)	64 (2^6)	32 (2^5)	16 (2^4)	8 (2^3)	4 (2^2)	2 (2^1)	1 (2^0)

$$128(1) + 64(0) + 32(0) + 16(1) + 8(0) + 4(1) + 2(1) + 1(0) = 226$$

Signed Bytes



$$(-1)(64(0) + 32(1) + 16(0) + 8(0) + 4(0) + 2(1) + 1(1)) = -43$$

Adding Unsigned Bytes (Overflow)

0	1	1	0	0	0	1	1	63_{10}
+								
1	1	0	0	1	0	1	1	203_{10}
=								
0	0	1	0	1	1	1	0	46_{10} $\neq$ 266_{10}

Adding Signed Bytes

0	1	1	0	0	0	1	1	63_{10}
+								
1	1	0	0	1	0	1	1	-75_{10}
=								
0	0	1	0	1	1	1	0	46_{10} $\neq$ -12_{10}

Two's Complement

0	1	1	0	0	0	1	1
---	---	---	---	---	---	---	---

63_{10}

Invert

1	0	0	1	1	1	0	0
---	---	---	---	---	---	---	---

+ 1

1	0	0	1	1	1	0	1
---	---	---	---	---	---	---	---

-63_{10}

Two's Complement

1	0	0	1	0	1	1	0
-128	64	32	16	8	4	2	1

$$-128(1) + 64(0) + 32(0) + 16(1) + 8(0) + 4(1) + 2(1) + 1(0) = -106$$

0	1	1	0	1	0	1	0
-128	64	32	16	8	4	2	1

$$-128(0) + 64(1) + 32(1) + 16(0) + 8(1) + 4(0) + 2(1) + 1(0) = 106$$

How To Write Big Numbers

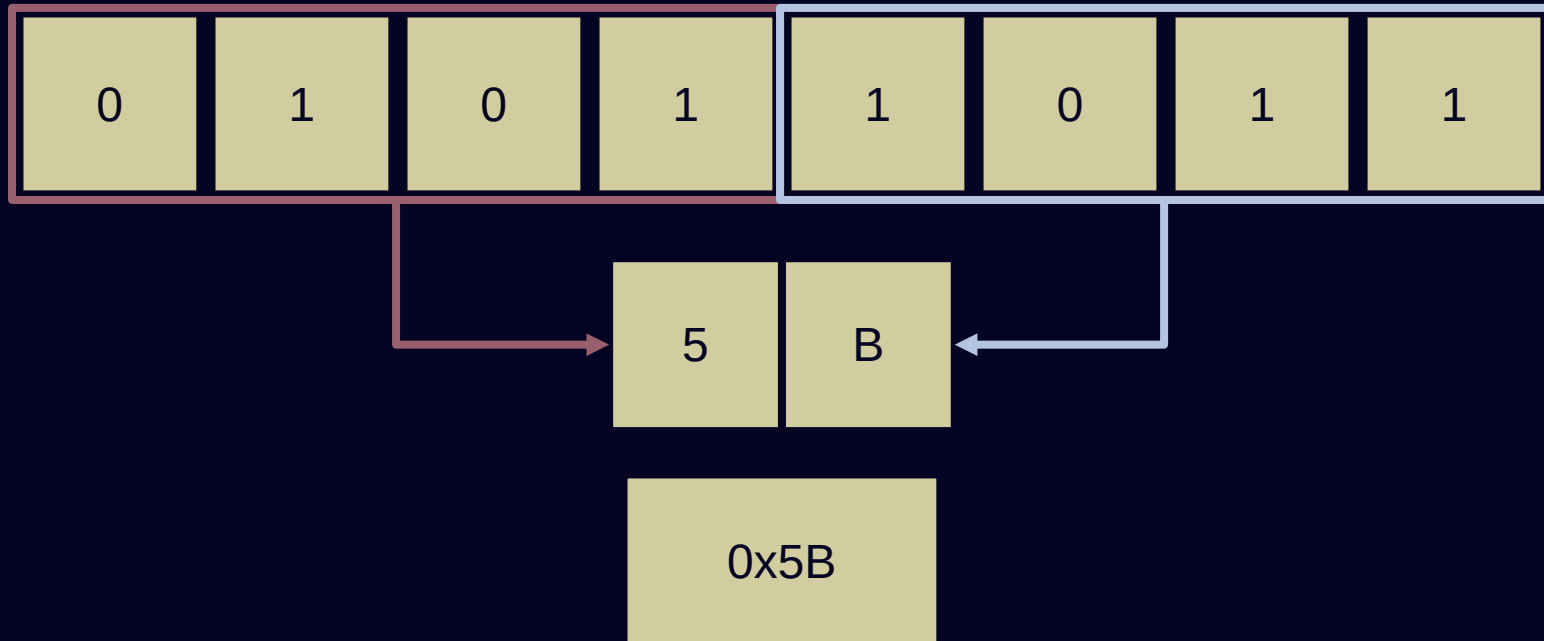
What if we want to represent the number 500_{10} ?



An unsigned byte can only represent up to 255_{10}

How To Write Big Numbers (Hexadecimal)

Numbers can be easily represented in hexadecimal (hex)



How To Write Big Numbers

500_{10}

0x01

0xF4

0xF4

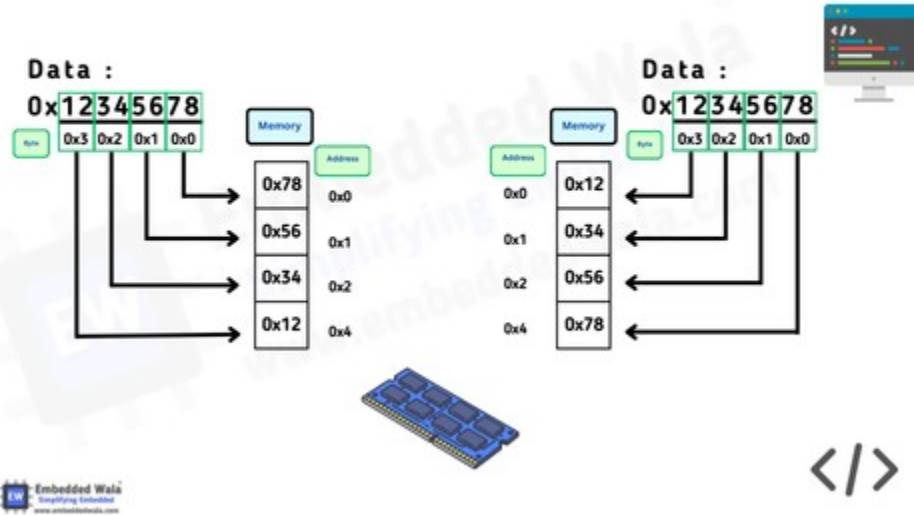
0x01

What's the right order to store this?

Endianness

It depends on the system!

Little Endian vs Big Endian



Endianness

500₁₀

Big Endian
(BE)

0x01

0xF4

Little Endian
(LE)

0xF4

0x01

Numeric Data Types

Unsigned Integers

- `uint8_t`
 - *unsigned char*
 - 0 to 255
- `uint16_t`
 - *unsigned short*
 - 0 to 65,535
- `uint32_t`
 - *unsigned long*
 - 0 to $2^{32}-1$
- `uint64_t`
 - *unsigned long long*
 - 0 to $2^{64}-1$

Signed Integers

- `int8_t`
 - *char*
 - -128 to 127
- `int16_t`
 - *short*
 - -32,768 to 32,768
- `int32_t`
 - *long*
 - -2^{31} to $2^{31}-1$
- `int64_t`
 - *long long*
 - -2^{63} to $2^{63}-1$

Floating Point

- `float`
 - 4 bytes
 - $\pm 3.402,823,4 \times 10^{38}$
 - About 7 digit precision
- `double`
 - 8 bytes
 - $\pm 1.797,693,134,862,315,7 \cdot 10^{308}$
 - About 15 digit precision

Boolean

- `bool` (true/false)
 - 1 byte

Declaring Variables

```
uint8_t a = 37;  
a = 52;
```

```
// 256 will not fit in a uint8_t  
// a = 256;
```

Arithmetic

```
int32_t b = 234;
```

```
// Addition
```

```
b = b + 25;
```

```
b += 25;
```

```
// Subtraction
```

```
b = b - 25;
```

```
b -= 25;
```

```
// Multiplication
```

```
b = b * 25;
```

```
b *= 25;
```

```
// Division
```

```
b = b / 25;
```

```
b /= 25;
```

```
// Remainder
```

```
b = b % 25;
```

```
b %= 25;
```

```
uint8_t c = 0b01101101;
```

```
// Bitwise NOT
```

```
c = ~c;
```

```
// Bitwise AND
```

```
c = c & 0x5F;
```

```
c &= 0x5F;
```

```
// Bitwise OR
```

```
c = c | 0x5F;
```

```
c |= 0x5F;
```

```
// Bitwise XOR
```

```
c = c ^ 0x5F;
```

```
c ^= 0x5F;
```

```
// Bitwise Left Shift
```

```
c = c << 5;
```

```
c <<= 5;
```

```
// Bitwise Right Shift
```

```
c = c >> 5;
```

```
c >>= 5;
```

Conditionals

Directs the flow of a program based on given conditions

```
bool condition_1 = true;
uint8_t condition_2 = 34;

if (condition_1) {
    // Do something
}
else if (condition_2 == 34) {
    // Do something else
}
else if (1 == 1 && 76 != 67) {
    // Do something else
}
else {
    // Do this if no other
    // conditions are true
}
```

```
uint8_t my_case = 2;
switch (my_case) {
    case 0:
        // Do something
        break;
    case 1:
        // Do something else
        break;
    case 2:
        // Do something else
    case 3:
    case 4:
        // Do something for both 3 or 4
        break;
    default:
        // Do this if no other cases are
        // true
        break;
}
```

Loops

While loops run “while” a condition is true

```
bool condition_3 = true;
while (condition_3) {
    // Loops as long as condition is true
}
```

```
bool condition_4 = true;
do {
    // Runs at least once
    // Loops as long as condition is true
} while (condition_4);
```

For loops run “for” a certain number of iterations

```
for (int i = 0; i < 5; i++) {
    // Loops until condition is true
    // Applies the given function each loop iteration
}
```

Loops can be exited at any time with a “break” statement (only exits immediate loop)

Functions

Functions are modular units of code

Functions must be unique

- Different names same parameters?
 - Different functions
- Same names different parameters?
 - Overloaded function
- Same name same parameters?
 - Error

```
// Function declaration tells your program the function
// exists. You can declare and define functions at the
// same time, or declare functions and define them later
uint64_t add_numbers(uint64_t num_1, uint64_t num_2);

void setup(){

}

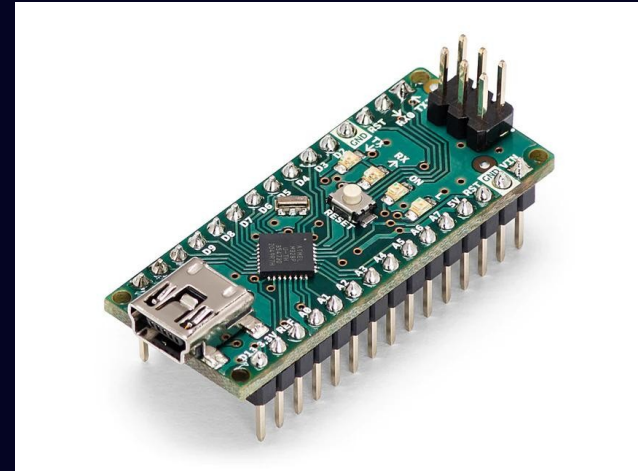
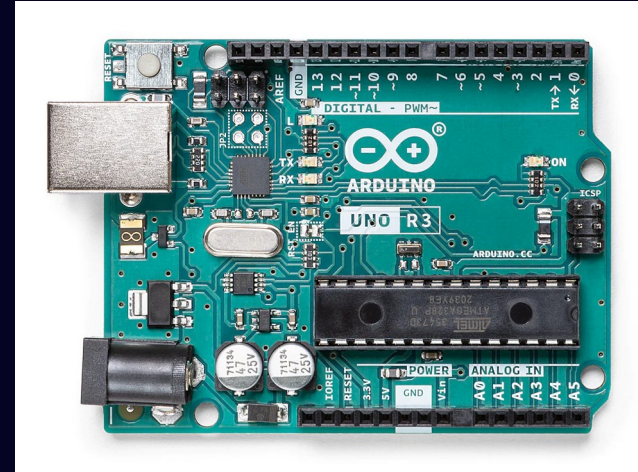
void loop(){
    // Function usage must match expected types
    uint64_t x = add_numbers(num_1: 34, num_2: 29);
}

// Function definition / implementation
uint64_t add_numbers(uint64_t num_1, uint64_t num_2) {
    // Some code
    return num_1 + num_2;
}
```

Working With Arduino

What is Arduino?

- Open-source hardware and software platform for building devices with single-board microcontrollers
- Cross-platform and easy to use
- Provides a straightforward set of instructions for interacting with the device's peripherals (digital I/O, analog I/O, SPI, I2C, etc.)



File Layout

- `#include <Arduino.h>`
 - Gives access Arduino-specific functions in the program
- `void setup() {}`
 - Program entry point
 - All code in this function runs once at startup
- `void loop() {}`
 - Runs immediately after setup
 - Loops forever

```
#include <Arduino.h>

// put function declarations here:
int myFunction(int, int);

void setup() {
    // put your setup code here, to run once:
    int result = myFunction(2, 3);
}

void loop() {
    // put your main code here, to run repeatedly:
}

// put function definitions here:
int myFunction(int x, int y) {
    return x + y;
}
```

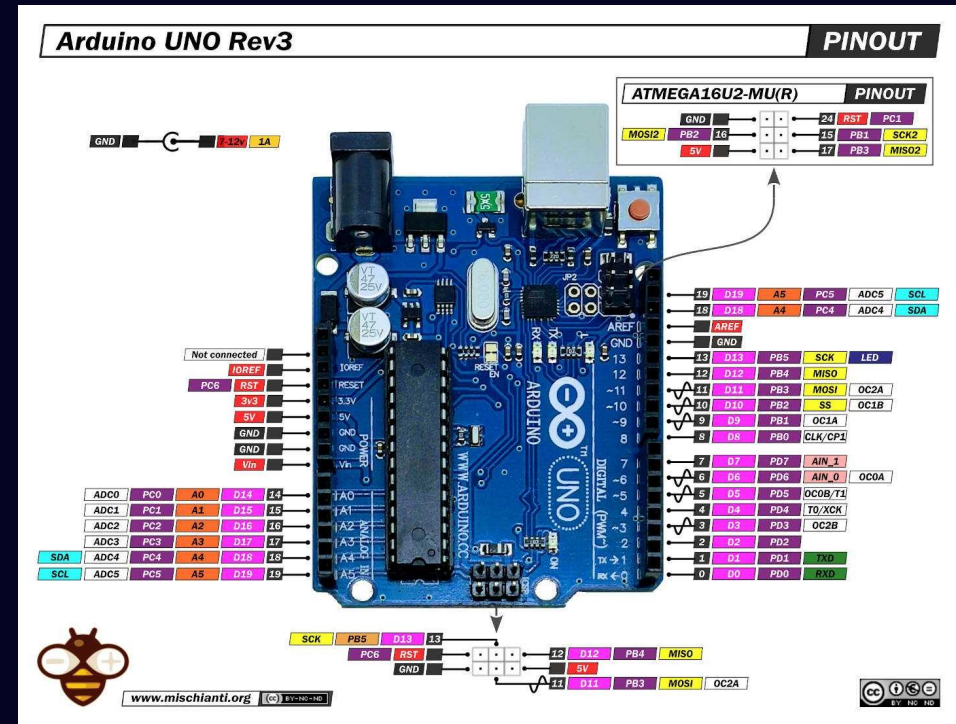
GPIO (General Purpose Input/Output)

GPIO are the pins on the microcontroller we can actually assign uses to!

The most common uses include:

- Driving outputs
- Communication protocols
- Reading inputs

Be aware of what pins do what, it differs per microcontroller



Basic Arduino Functions

Arduino provides many functions to control the hardware and peripherals of the microcontroller

- **delay(ms)**
 - Halts the program for the specified number of milliseconds
- **delayMicroseconds(us)**
 - Halts the program for the specified number of microseconds
- **millis()**
 - Returns the number of milliseconds since the current program started
- **pinMode(pin, mode)**
 - Initializes a pin to the specified mode
 - INPUT, OUTPUT, INPUT_PULLUP
- **digitalWrite(pin, value)**
 - Drives a pin high or low
 - HIGH, LOW
- **digitalRead(pin)**
 - Reads the state of a pin

Lab!

Git Basics

Common Git Commands

- **git init**
 - Creates a new git repo in the current directory
- **git clone <url> <*file>**
 - Copies a remote repo locally
 - Optional output file name
- **git add <*>**
 - Adds files to commit staging area
 - To add all files use “.”
 - Required before committing
- **git commit -m “message”**
 - Creates a new commit (save point) with a message
- **git push**
 - Sends local repo changes to remote
- **git pull**
 - Updates local repo from remote
- **git checkout <-b> <branch>**
 - Switches branches
 - Add “-b” to create new branch
- **git merge <branch>**
 - Combines “branch” with current branch
 - Good luck with merge conflicts!
- **git restore <*>**
 - Resets specified files to previous commit
 - To reset all files use “.”
- **git remote add origin <url>**
 - Links local repo to remote repo

Software to Install

Software Installation

- VSCode (<https://code.visualstudio.com/download>)
 - Extensions:
 - Platformio (<https://marketplace.visualstudio.com/items?itemName=platformio.platformio-ide>)
 - Wokwi (<https://marketplace.visualstudio.com/items?itemName=Wokwi.wokwi-vscode>)
- Git (<https://git-scm.com/install/>)

Blink the LED!

Instructions are included in the lab file, but TLDR: Make the built-in LED blink!

- Find a suitable folder to clone the training repo
- Run the following commands in the terminal to clone the repo and get started!
 - `git clone https://github.com/NEURoboticsClub/firmware-course.git`
 - `cd firmware-course`
 - `git checkout -b "yourgitusername-training"`

Let me know if you need help!